Concept 7: Relational vs Non-Relational Databases (SQL vs NoSQL)

Why This Matters

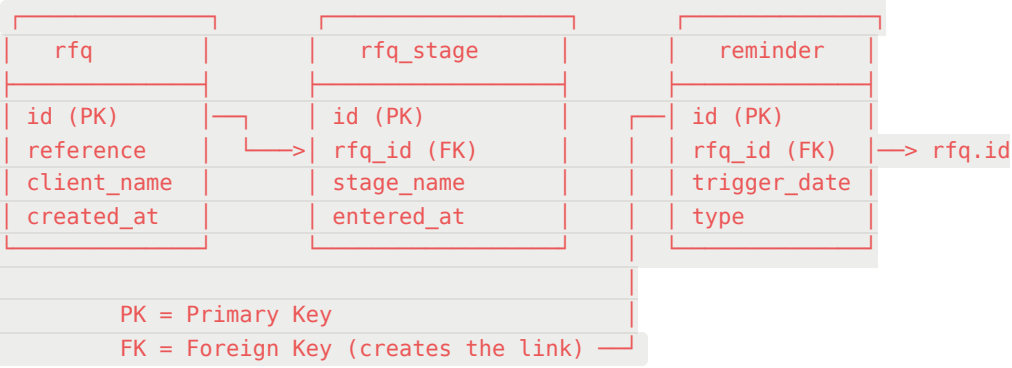
The database is the foundation of every service. Choosing the wrong type means fighting the technology instead of leveraging it. Your boss brought this up because different parts of the RFQ platform may benefit from different database types — and he wants you to understand WHY, not just pick one.

Relational Databases (SQL)

Examples: PostgreSQL, MySQL, SQL Server, Oracle

Core principle: Data is organized in **tables** (rows and columns) with **strict schemas** defined upfront. Relationships between tables are explicit via **foreign keys**.

How It Works



You define the schema BEFORE inserting data:

sql

```
CREATE TABLE rfq (
    id SERIAL PRIMARY KEY,
    reference VARCHAR(50) NOT NULL,
    client_name VARCHAR(100) NOT NULL,
```

```
    created_at TIMESTAMP DEFAULT NOW()
);
```

Every row must conform to this structure. You can't insert an RFQ without a reference — the database enforces it.

Key Properties

ACID Transactions:

- **Atomicity:** All operations in a transaction succeed or all fail (no partial updates)
- **Consistency:** The database always moves from one valid state to another
- **Isolation:** Concurrent transactions don't interfere with each other
- **Durability:** Once committed, data survives crashes

This is CRITICAL for RFQ operations. When you move an RFQ to a new stage, you need:

1. Update the RFQ status
2. Create a new stage record
3. Log the transition

If step 2 fails, you want ALL steps to roll back. SQL gives you this guarantee.

Joins: SQL lets you combine data from multiple tables in a single query:

sql

```
SELECT r.reference, s.stage_name, rem.trigger_date
FROM rfq r
JOIN rfq_stage s ON s.rfq_id = r.id
LEFT JOIN reminder rem ON rem.rfq_id = r.id
WHERE r.id = 42;
```

One query, three tables, complete picture. This is the power of relational design.

Non-Relational Databases (NoSQL)

Examples: MongoDB (document), Redis (key-value), Cassandra (column), Neo4j (graph)

There are several types, but MongoDB (document store) is what your boss mentioned, so we'll focus there.

MongoDB — Document Store

Core principle: Data is stored as flexible **documents** (JSON-like objects called BSON). No predefined schema — each document can have different fields.

json

```
// A single MongoDB document for an RFQ
{
  "_id": "ObjectId('abc123')",
  "reference": "RFQ-2025-042",
  "client": {
    "name": "Saudi Aramco",
    "contact": "Ahmed",
    "email": "ahmed@aramco.sa"
  },
  "stages": [
    { "name": "Reception", "entered_at": "2025-01-15", "completed_at": "2025-01-16" },
    { "name": "Technical Review", "entered_at": "2025-01-16", "completed_at": null }
  ],
  "reminders": [
    { "type": "internal", "trigger_date": "2025-01-20", "message": "Follow up" }
  ],
  "line_items": [
    { "description": "Pressure Vessel", "quantity": 2, "material": "SA-516 Gr.70" }
  ]
}
```

Notice: everything is **nested inside one document**. No joins needed — you fetch one document and you have everything about that RFQ.

Key Properties

Schema flexibility: You can add fields to any document without altering a schema. One RFQ can have `urgency_level` while another doesn't.

Denormalization: Instead of splitting data across tables and joining, you embed related data directly. This makes reads fast (one query = complete document) but makes updates harder (if client info changes, you update it everywhere it's embedded).

Horizontal scaling: MongoDB is designed to distribute data across many servers (sharding). SQL databases scale vertically (bigger server) more naturally.

Eventual consistency (by default): In distributed setups, a write might not be immediately visible on all replicas. You can configure stronger consistency, but it's not the default like SQL.

Head-to-Head Comparison

Criteria	SQL (PostgreSQL)	NoSQL (MongoDB)
Schema	Strict, predefined	Flexible, per-document
Relationships	Explicit (foreign keys + joins)	Implicit (embedding or manual references)
Transactions	Full ACID	ACID on single document; multi-doc transactions available but less mature
Query power	Very powerful (SQL language, aggregations, window functions)	Good for document queries, aggregation pipeline is powerful but different
Data integrity	Enforced by database (NOT NULL, UNIQUE, FK constraints)	Enforced by application code
Scaling	Vertical (bigger server) + read replicas	Horizontal (sharding across servers)
Best for	Structured data with clear relationships	Flexible/varied data, rapid prototyping, document-centric storage
Worst for	Rapidly changing schemas, massive scale	Complex multi-table joins, strict integrity requirements

ORM — Object-Relational Mapping

An **ORM** is a library that lets you interact with the database using your programming language's objects instead of writing raw SQL or query syntax.

For SQL (Python): SQLAlchemy

python

```
# Instead of writing SQL:
# SELECT * FROM rfq WHERE status = 'active'
```

```
# You write:
active_rfqs = session.query(RFQ).filter(RFQ.status == 'active').all()
```

For MongoDB (Python): MongoEngine or PyMongo

python

```
# Instead of writing MongoDB queries:
# db.rfqs.find({"status": "active"})

# With MongoEngine you write:
active_rfqs = RFQ.objects(status='active')
```

For MongoDB (Node.js): Mongoose

javascript

```
const activeRfqs = await RFQ.find({ status: 'active' });
```

Why it matters: The ORM maps your code's classes/objects directly to database tables (SQL) or collections (MongoDB). It abstracts the database syntax, making your code database-agnostic to some degree. But it can also hide performance issues — complex ORM queries sometimes generate inefficient SQL under the hood.

The Benchmark — How to Choose

Your boss mentioned doing a **benchmark**. This doesn't mean a speed test — it means systematically evaluating both options against YOUR project's criteria. Here's the framework:

Criteria to Evaluate

Criterion	Question to Answer	SQL Advantage?	MongoDB Advantage?
Data structure	Is our data highly relational?	✅ RFQs have clear relations (stages, items, reminders)	
Schema stability	Will our schema change frequently?	✅ RFQ structure is well-defined	Could help for historical/varied data
Integrity needs	Do we need strict constraints?	✅ RFQ state transitions must be consistent	

Criterion	Question to Answer	SQL Advantage?	MongoDB Advantage?
Query patterns	Do we need complex joins?	✅ Dashboards, reports need multi-table queries	
Flexibility	Do we need varied document structures?		✅ Historical RFQs may vary in structure
Scale	Will we have millions of records?	Sufficient for GHI's volume	Designed for massive scale
Team expertise	What does the team know?	✅ BACAB has SQL experience	Learning opportunity
Ecosystem	What integrates well?	✅ Mature tooling, ORMs, migration tools	Growing ecosystem

How to Present the Benchmark

Structure it as: "For criteria X, option A scores better because [reason], while option B scores better for criteria Y because [reason]. Given our project's priorities, we recommend [choice]."

Key Vocabulary

Term	Definition
Schema	The structure definition of your data (what columns/fields exist, their types, constraints)
Foreign Key	A column that references the primary key of another table, creating a relationship
Join	A SQL operation that combines rows from two or more tables based on a related column
ACID	Atomicity, Consistency, Isolation, Durability — guarantees of relational transactions
Document	In MongoDB, a single JSON-like record (equivalent to a row in SQL, but flexible)
Collection	In MongoDB, a group of documents (equivalent to a table in SQL)
Denormalization	Storing related data together (embedding) instead of in separate tables

Term	Definition
Sharding	Distributing data across multiple servers for horizontal scaling
ORM	A library that maps code objects to database structures
Eventual Consistency	A model where reads might not immediately reflect the latest write across all replicas